

state_machine.py — teoria i opis modułu maszyny stanów

1. Rola modułu w systemie Stop & Go

Moduł `state_machine.py` jest centrum decyzyjnym całego systemu. Przyjmuje jedną liczbę — odległość d w metrach z `distance.py` — i na jej podstawie decyduje w jakim trybie ma pracować pojazd. Wynikiem każdego wywołania jest jeden z czterech stanów: JAZDA, ZWALNIANIE, STOJ lub RUSZANIE. Ten stan trafia do `controller.py` który przelicza go na konkretną wartość PWM silnika.

Moduł implementuje klasyczny automat skończony (Finite State Machine, FSM) — jeden z fundamentalnych modeli obliczeniowych teorii automatów. FSM jest powszechnie stosowany w systemach wbudowanych, robotyce i systemach ADAS (Advanced Driver Assistance Systems) wszędzie tam gdzie system musi reagować na zdarzenia dyskretne w przewidywalny i weryfikowalny sposób.

2. Teoria automatów skończonych — FSM

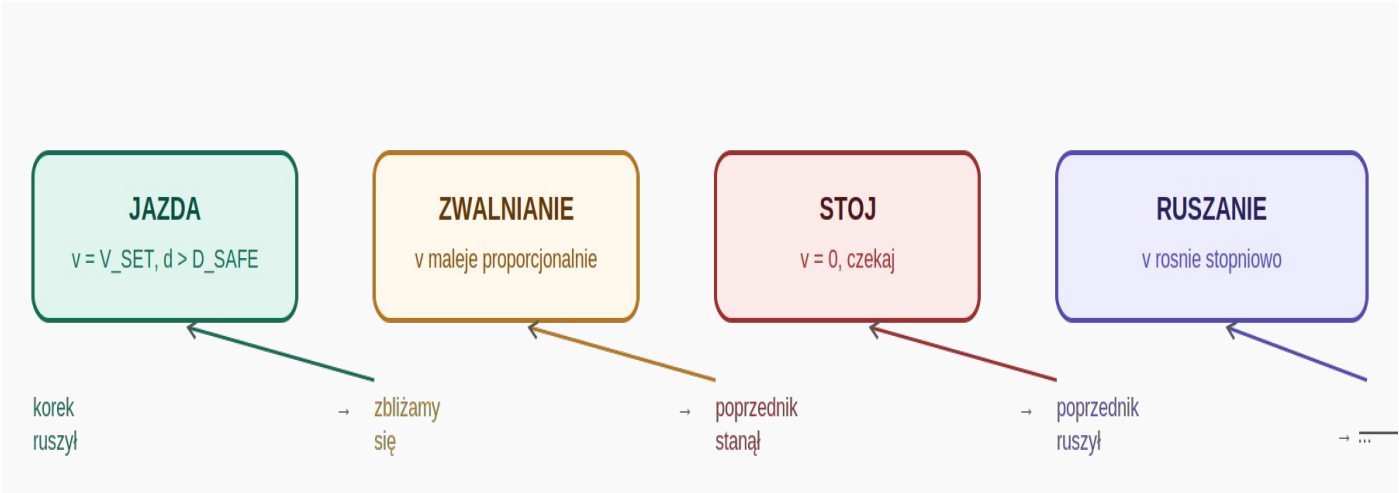
Automat skończony (FSM) jest formalnie definiowany przez piątkę $(Q, \Sigma, \delta, q_0, F)$ gdzie Q to skończony zbiór stanów, Σ to alfabet wejściowy (zbiór możliwych wejść), δ to funkcja przejść, q_0 to stan początkowy, a F to zbiór stanów akceptujących. W naszym projekcie:

Q (stany)	{ JAZDA, ZWALNIANIE, STOJ, RUSZANIE }
Σ (wejście)	odległość $d \in \mathbb{R}_{\geq 0}$ lub None (brak pomiaru)
δ (funkcja przejść)	<code>_from_jazda()</code> , <code>_from_zwalnianie()</code> , <code>_from_stoj()</code> , <code>_from_ruszanie()</code>
q_0 (stan startowy)	JAZDA — pojazd gotowy do jazdy
F (stany końcowe)	brak — system działa w pętli nieskończonej

W odróżnieniu od klasycznego FSM (gdzie wejście jest symbolem z alfabetu), nasz automat jest FSM z wartościowymi wejściami — wejście to liczba rzeczywista porównywana z progami. Takie automaty są nazywane Extended Finite State Machines (EFSM) i są standardem w systemach sterowania.

3. Uzasadnienie czterech stanów

Minimalna liczba stanów do implementacji funkcji Stop & Go wynosi dwa: JEDZ i STOJ. Jednak taki system miałby poważne wady. Dodanie stanów ZWALNIANIE i RUSZANIE jest uzasadnione zarówno z perspektywy teorii sterowania jak i wymagań bezpieczeństwa.



Rysunek 1. Cztery stany odpowiadają czterem fizycznym fazom ruchu pojazdu w korku miejskim.

3.1 Stan ZWALNIANIE — proporcjonalne prawo sterowania

Bezpośrednie przejście JAZDA → STOJ oznaczałoby skok prędkości z V_SET do 0 w jednej klatce (33 ms). Jest to nieakceptowalne z kilku powodów: przeciążenie mechaniki napędu, brak komfortu jazdy, oraz w rzeczywistym pojeździe — ryzyko poślizgu tylnego pojazdu.

Stan ZWALNIANIE implementuje proporcjonalne prawo sterowania: prędkość spada liniowo wraz ze zmniejszaniem się odległości między progami D_SLOW i D_STOP . To jeden z podstawowych elementów regulacji PID — człon proporcjonalny (P) reaguje na uchyb (różnicę między aktualną a pożądaną odległością) proporcjonalną zmianą sterowania.

Dlaczego proporcjonalne, nie stałe zwalnianie?

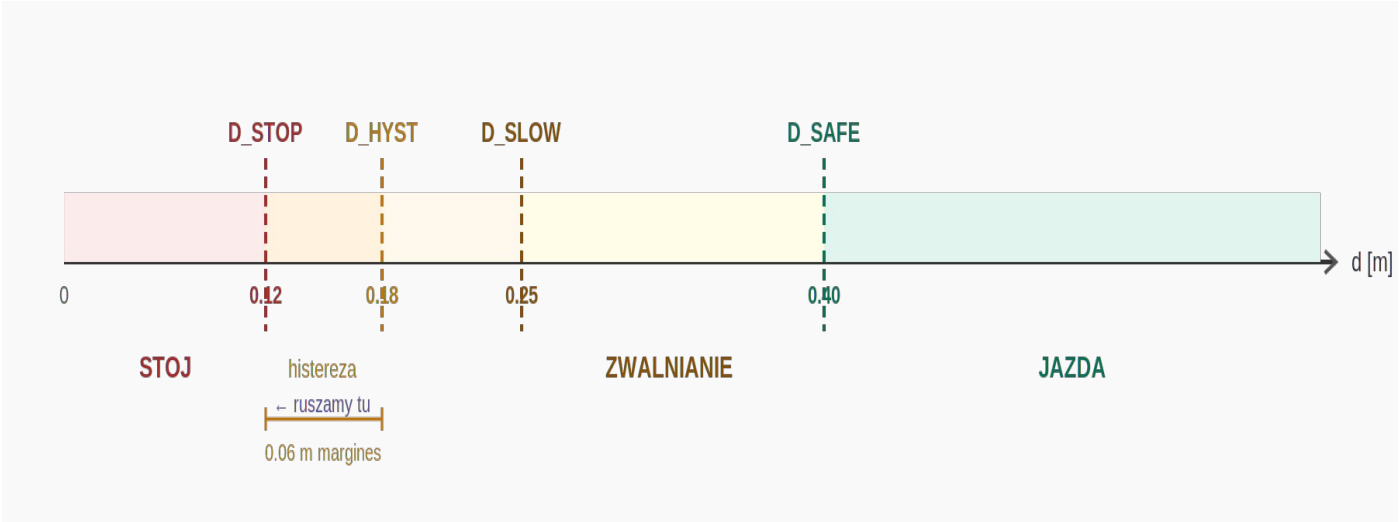
Stale zwalnianie (zawsze -X% mocy w strefie ZWALNIANIE) nie uwzględnia dynamiki: przy zbliżaniu z dużą prędkością hamowanie musi być silniejsze niż przy powolnym zbliżaniu. Regulacja proporcjonalna automatycznie dostosowuje intensywność hamowania do aktualnej odległości.

3.2 Stan RUSZANIE — kontrolowane przyspieszanie

Symetrycznie do ZWALNIANIE, stan RUSZANIE zapobiega gwałtownemu skokowi z $v=0$ do V_SET . W systemach ADAS ten etap jest opisywany jako smooth departure i jest wymagany przez normy bezpieczeństwa (ISO 15622 dla ACC, ISO 22179 dla FSRA).

Praktyczny powód dla modelu RC: silnik DC ma nieliniarną charakterystykę — przy nagłym podaniu pełnego napięcia pobiera duży prąd rozruchowy co może resetować Raspberry Pi jeśli zasilanie nie jest wystarczająco wydajne. Stopniowe przyspieszanie przez RUSZANIE eliminuje ten problem.

4. Progi odległości i ich rozmieszczenie



Rysunek 2. Rozmieszczenie progów na osi odległości. Strefa histerezy między D_STOP a D_HYST zapobiega drganiom systemu.

$D_SAFE = 0.40\text{ m}$	Setpoint systemu — pożądana odległość podróżna. Powyżej tej wartości pojazd jedzie ze stałą prędkością V_SET . Odpowiada ~4 m w skali rzeczywistej (model 1:10).
$D_SLOW = 0.25\text{ m}$	Dolna granica strefy zwalniania. Wejście w strefę $[D_SLOW, D_SAFE]$ inicjuje proporcjonalne hamowanie. Nazwa mówi: od tej odległości zaczynamy zwalniać.
$D_STOP = 0.12\text{ m}$	Próg awaryjnego zatrzymania. Poniżej tej wartości $v=0$ niezależnie od innych warunków. Wyznaczany eksperymentalnie — musi uwzględniać opóźnienie mechaniczne hamulca.
$D_HYST = 0.18\text{ m}$	Próg histerezy. Pojazd rusza dopiero gdy d przekroczy tę wartość. Musi być $D_STOP < D_HYST < D_SLOW$. Margines $D_HYST - D_STOP = 0.06\text{ m}$ pochłania szum pomiarowy.

5. Histereza — eliminacja drgań na granicy decyzji

Histereza (od gr. *hystérēsis* — opóźnienie) to właściwość systemu polegająca na tym że zachowanie systemu zależy nie tylko od aktualnej wartości wejścia, ale też od historii — w szczególności od kierunku zmiany. W elektronice histereza jest używana np. w wyzwalaczach Schmitta do eliminacji drgań przy powolnych zmianach sygnału wejściowego.

W naszym systemie histereza jest zaimplementowana asymetrycznie: wchodzimy do STOJ gdy $d \leq D_STOP$ (0.12 m), ale wychodzimy ze STOJ dopiero gdy $d > D_HYST$ (0.18 m). Próg wyjścia jest wyższy niż próg wejścia — to jest definicja histerezy.



Rysunek 3. Efekt histerezy: bez niej szum pomiarowy ± 0.01 m wokół D_STOP powoduje chattering. Z histerezą stan zmienia się stabilnie tylko gdy d naprawdę przekroczy D_HYST .

5.1 Obliczenie minimalnego marginesu histerezy

Margines histerezy musi być większy niż amplituda szumu pomiarowego. Z `distance.py` wiemy że:

```
Szum  $W\_pixel$  z detektora:  $\pm 3$  px (typowo przy KCF)
Przy  $d=0.12$  m,  $W\_pixel \approx 400$  px:
 $dd = d * (dw/W) = 0.12 * (3/400) = 0.0009$  m  $\approx 0.1$  cm

Po wygładzeniu medianą (window=5):
szum efektywny  $\approx 0.002$ - $0.005$  m

Margines histerezy:  $D\_HYST - D\_STOP = 0.18 - 0.12 = 0.06$  m
>> szum  $0.005$  m  $\rightarrow$  margines 12× większy niż szum  $\rightarrow$  stabilne
```

Histereza na poziomie stanów (ZWALNIANIE $\leftrightarrow$ JAZDA):

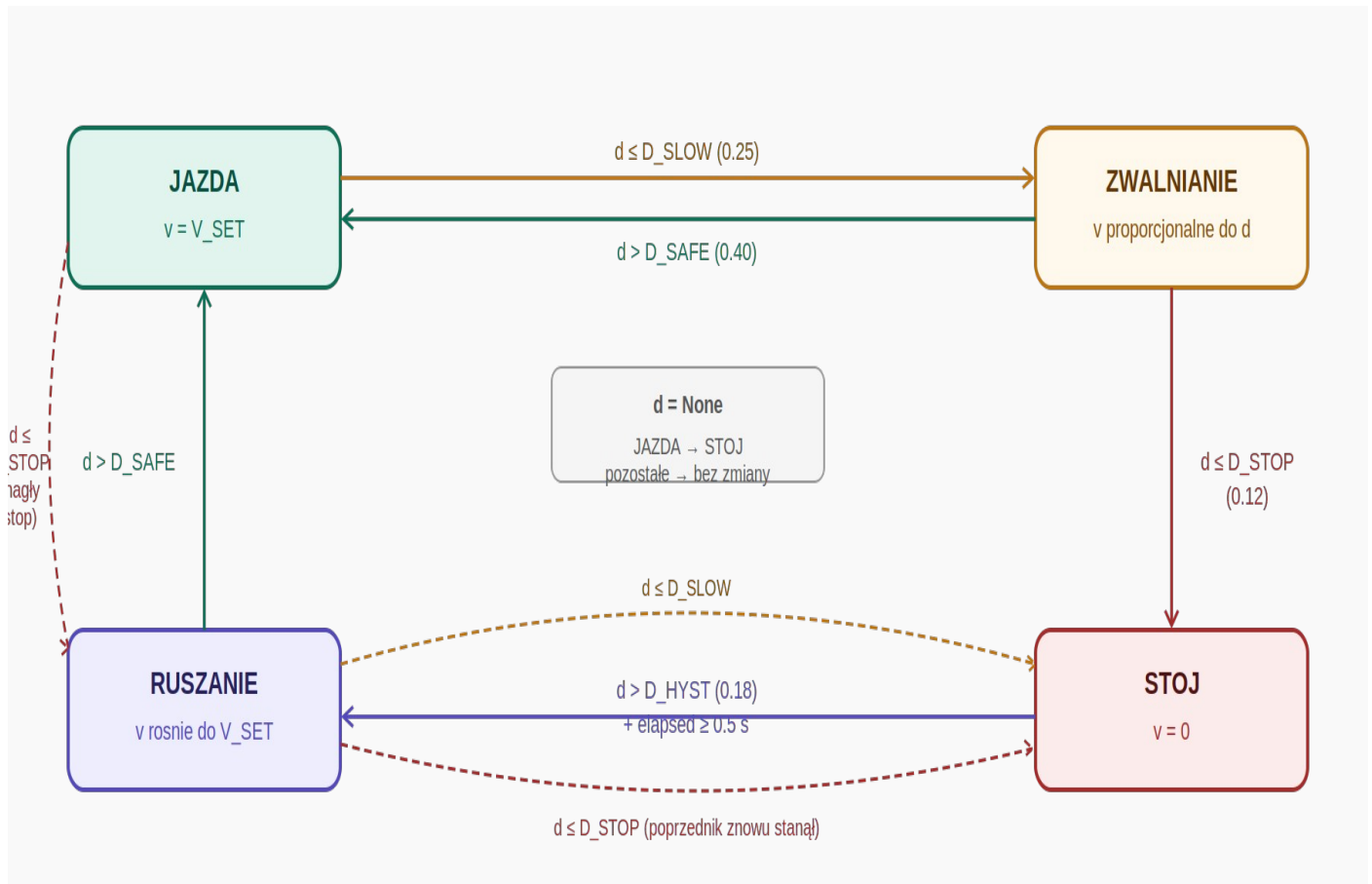
Podobna zasada obowiązuje przy przejściu JAZDA $\leftrightarrow$ ZWALNIANIE.

Do ZWALNIANIE wchodzimy przy $d \leq D_SLOW$ (0.25 m),
ale z ZWALNIANIE wracamy do JAZDA dopiero przy $d > D_SAFE$ (0.40 m).

Margines: $D_SAFE - D_SLOW = 0.40 - 0.25 = 0.15$ m.

Bez tego pojazd oscylowałby między JAZDA a ZWALNIANIE
przy jeździe z prędkością dokładnie na granicy D_SLOW .

6. Kompletna maszyna stanów — wszystkie przejścia



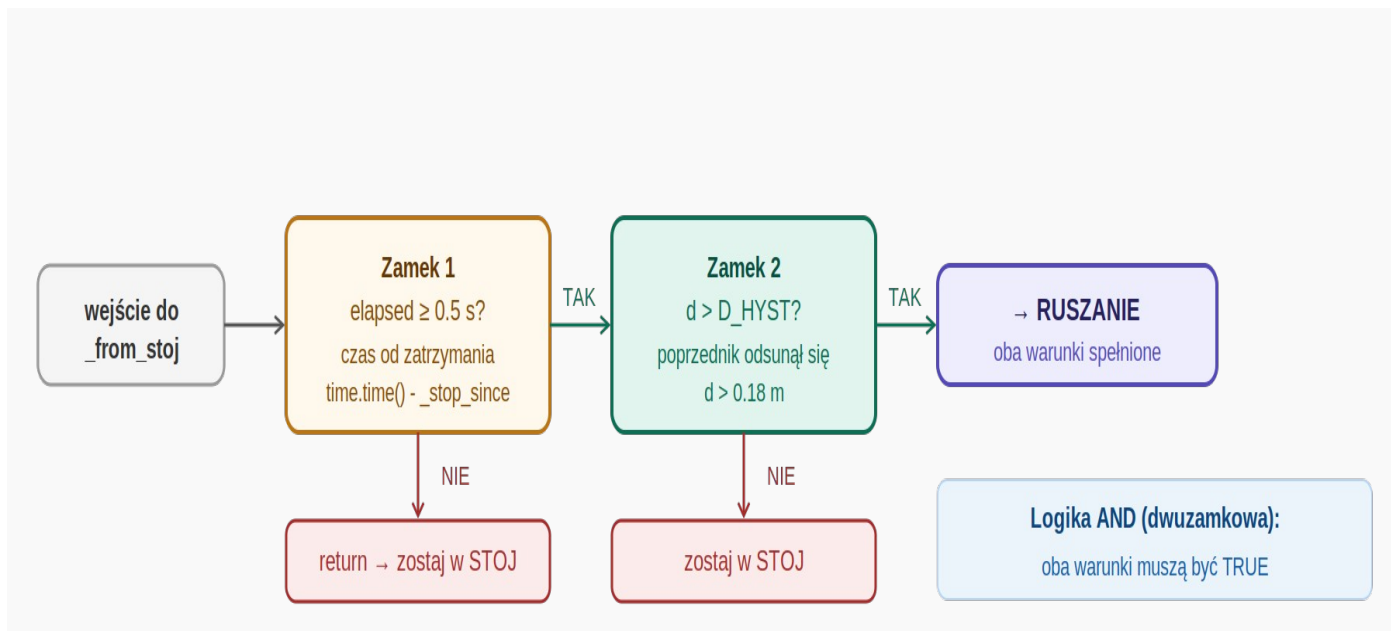
Rysunek 4. Kompletna maszyna stanów z wszystkimi możliwymi przejściami. Przejście JAZDA → STOJ (przerywana linia) to emergency stop przy nagłym hamowaniu poprzednika.

Warto zwrócić uwagę na przejście JAZDA → STOJ (linia przerywana po lewej stronie diagramu). To tzw. emergency stop — sytuacja gdy poprzednik zahamował gwałtownie i odległość spadła poniżej D_STOP bez przechodzenia przez strefę ZWALNIANIE. System reaguje natychmiast z pominięciem fazy ZWALNIANIE. Jest to celowe — bezpieczeństwo ma priorytet nad komfortem.

Stan RUSZANIE ma pełny zakres przejść — może wrócić do STOJ (poprzednik zatrzymał się ponownie w trakcie naszego ruszania), przejść do ZWALNIANIE (zbliżamy się za szybko), lub osiągnąć JAZDA (ruszanie zakończone sukcesem). Obsługuje to scenariusz stop-start typowy dla korków miejskich.

7. Logika dwuzamkowa — warunki wyjścia ze stanu STOJ

Przejście STOJ → RUSZANIE jest jedynym przejściem w całym FSM które wymaga spełnienia dwóch niezależnych warunków jednocześnie. Jest to świadoma decyzja projektowa wynikająca z wymagań bezpieczeństwa.



Rysunek 5. Logika dwuzamkowa przy wyjściu ze stanu STOJ. Oba warunki muszą być spełnione jednocześnie — analogia do zamka z dwoma kluczami.

7.1 Warunek czasowy — STOP_HOLD_TIME_S

Minimalny czas oczekiwania w stanie STOJ (0.5 s) chroni przed fałszywym wyjściem ze stanu STOJ wynikającym z krótkotrwałej utraty obrazu lub błędnego pomiaru odległości.

Scenariusz bez STOP_HOLD_TIME_S:

```

klatka 1: d = 0.10 m → STOJ (poprzednik stanął)
klatka 2: d = None → zostajemy w STOJ (brak pomiaru)
klatka 3: d = None → zostajemy w STOJ
klatka 4: d = 0.25 m → d > D_HYST → RUSZANIE ← fałszywy pomiar!

```

Scenariusz z STOP_HOLD_TIME_S = 0.5 s:

```

klatka 4 (t=0.13 s): elapsed = 0.13 s < 0.5 s → return, zostaj w STOJ
klatka 15 (t=0.50 s): elapsed = 0.50 s ≥ 0.5 s → sprawdź odległość

```

7.2 Warunek odległościowy — D_HYST

Drugi zamek sprawdza odległość dopiero po przepuszczeniu przez zamek czasowy. Separacja tych dwóch sprawdzeń przez dwa osobne instrukcje return jest celowa — każdy zamek jest niezależny. Nawet jeśli zamek 1 przepuszcza (minęło 0.5 s), zamek 2 może zatrzymać (poprzednik jeszcze za blisko). Logika AND jest realizowana przez sekwencyjne wyjście z funkcji.

Analogia do systemów bezpieczeństwa:

Logika dwuzamkowa jest standardem w systemach bezpieczeństwa krytycznego.

Przykłady: reaktory jądrowe (dwa niezależne systemy chłodzenia), awionika (dwa piloci muszą potwierdzić manewr), systemy ADAS (potwierdzenie z radaru I kamery przed aktywacją hamowania).

W naszym projekcie skala jest mniejsza, ale zasada identyczna:

dwa niezależne kryteria muszą być spełnione przed ruszeniem pojazdu.

8. Obsługa braku pomiaru — d = None

Przypadek gdy distance.py zwraca None (brak detekcji pojazdu poprzedzającego) wymaga osobnej polityki zachowania. Nie istnieje jednoznacznie poprawna odpowiedź — każda strategia ma swoje kompromisy. Implementujemy strategię state-dependent (zależną od stanu):

JAZDA + d=None	Przejdźcie do STOJ. Nie wiemy co jest przed nami — bezpieczniej zatrzymać się niż jechać w ciemno. Strategia fail-safe.
ZWALNIANIE + d=None	Pozostajemy w ZWALNIANIE. Kontynuujemy spowalnianie. Krótkotrwała utrata obrazu podczas hamowania nie powinna przerywać manewru.
STOJ + d=None	Pozostajemy w STOJ. Nie wiemy czy poprzednik ruszył — nie ruszamy. Strategia fail-safe.
RUSZANIE + d=None	Pozostajemy w RUSZANIE. Krótkotrwała utrata obrazu podczas ruszania nie powinna zatrzymywać pojazdu.

Dlaczego nie zawsze STOJ przy d=None?

Kamera może tracić obraz na 1-3 klatki przy przesłonięciu lub ruchu. Jeśli byliśmy w RUSZANIE i na 2 klatki (66 ms) stracimy obraz, zatrzymanie pojazdu a potem ponowne ruszanie byłoby niepotrzebnym szarpnięciem. Strategia hold (utrzymuj stan) jest bardziej naturalna dla krótkich przerw w pomiarze.

9. Interfejs publiczny

update(distance_m)	Główna metoda. Przyjmuje float None, zwraca State. Wywołać raz na każdą klatkę w pętli głównej — ta sama częstotliwość co camera i detector.
state	Property (odczyt). Aktualny stan jako State enum. Używany przez controller.py.
changed	Property (odczyt). True jeśli stan zmienił się w ostatnim update(). Używany przez main.py do warunkowego logowania.

10. Odniesienia literaturowe

- Kaehler A., Bradski G. — Learning OpenCV 3, O'Reilly 2016.
Kontekst wizji komputerowej dostarczającej dane wejściowe d do FSM.
- Shirpour M. et al. — Traffic Object Detection Based on the Attentional Visual Field of Drivers. IEEE Transactions on Intelligent Vehicles, 2023.
Opis systemów ADAS i wymagań dotyczących reagowania na zatrzymanie pojazdu poprzedzającego.
- ISO 15622:2018 — Intelligent transport systems — Adaptive cruise control systems.
Norma definiująca wymagania dla systemów ACC w tym smooth departure i progi bezpieczeństwa.
- Hopcroft J.E., Motwani R., Ullman J.D. — Introduction to Automata Theory, Languages, and Computation. Pearson, 2006.
Formalna definicja automatów skończonych (FSM) i Extended FSM stosowanych w systemach sterowania.